TQMS Checkpoint Portal: System Analysis

Prepared for the Frontend Developer Technical Assessment | 8 November 2026

Executive summary

The Transport and Queue Management System (TQMS) supports work at a checkpoint: staff identify a vehicle, record its route and load, move it through checkpoint states, and issue or review a receipt. Around that central task are supporting records for registered vehicles, vehicle owners, checkpoint staff, lookup lists, reports and QR based entry.

This project recreates the checkpoint-facing workflows as a modern web portal. The Next.js frontend is organized by business feature and communicates with a separate NestJS API. The API starts with seeded demo data, can keep data in memory for local development, and can persist application collections to a configured Neon PostgreSQL database. Uploaded documents are sent from the backend to Cloudinary; the database stores their delivery references. This makes the API useful for a deployed demonstration, while the storage and authentication design still needs work before it should be treated as a production system.

The design follows a practical boundary: screens depend on an API contract rather than on database details. That keeps the interface replaceable when a real, supported backend becomes available. It also makes the current limitations visible instead of implying that the mock implementation is equivalent to the original system.

1. Scope and method

The supplied technical assignment asks the developer to understand an existing enterprise workflow, then build a maintainable frontend with a realistic mock API. Its evaluation points include feature based architecture, reusable forms and tables, search and pagination, validation, error and loading states, and replaceable API integration.

This analysis describes two related things: the checkpoint workflows visible in the reference portal and the behavior implemented in this project. The reference interface informed the journeys and vocabulary. The project source code and API contract define what the rebuilt application actually does. When those differ, this report calls out the difference rather than assuming that a visible control proves a particular server behavior.

The work is limited to observation and implementation analysis. It does not claim to describe the complete original TQMS backend or its database schema. The demo account and records are local sample data; they are not authorization to transact against the live service.

2. Users and system boundary

The primary user is checkpoint staff. Their main task is operational: register an arriving vehicle, verify the journey information, record a weight at the appropriate point, and complete the journey. The portal also provides administrative tasks such as managing checkpoint users, vehicle records and owners, and viewing reports.

At a high level, the system has three application parts:

1. Browser UI: Next.js pages and feature components render forms, lists, details, receipts, reports and QR output.
2. Application API: NestJS controllers enforce route-level behavior, apply request parsing and validation, handle sessions, upload files and produce consistent response shapes.
3. Persistence and media: the in-process demo collections are either held in memory or loaded from Neon as JSONB collection documents; Cloudinary stores uploaded media and returns URLs for the API to retain with records.

The browser does not connect directly to Neon or Cloudinary. This is an important security and maintainability boundary: credentials for those services belong only in the backend environment. The frontend receives data and media references through the API.

3. Main navigation and user journeys

The portal groups work into Dashboard, Checkpoints, Queue, Vehicle, Report, Generate QR and account/profile actions. The exact menu presentation differs between the observed reference and this implementation, but the same core checkpoint tasks are represented.

3.1 Queue entry and processing

Queue work is the central operational journey. A staff member opens Add new queue, chooses the vehicle category, selects or enters vehicle information, chooses a starting point and destination, and enters weight, date, transport company, load details and narration. A receipt preview updates while the form is being completed, allowing the operator to check the information before saving.

Registered vehicles are selected from existing vehicle records. Indian or unregistered vehicles can be entered through the alternate vehicle flow. Once saved, queue records move through explicit states such as in queue, in job, payment pending, or unregistered. The active queue list can be narrowed by operational attributes. Check-in and end-journey actions are modeled as separate transitions; they are not treated as arbitrary edits to a row. Pending QR submissions have their own review, approve and remove actions.

This separation is deliberate. A lifecycle action carries meaning for the checkpoint and can affect receipt or queue state, so it should be presented as a clear action with its own loading and result feedback. The receipt preview is a convenience for checking a draft; final values such as receipt number may only be available after the API creates a record.

3.2 Vehicle registration and detail

Staff can search vehicles, add or edit a registered vehicle, and inspect a detail page. Registration fields depend on the selected plate format. The form supports different plate types and derives the displayed plate from the supplied components, reducing formatting mistakes.

The vehicle detail area groups related information: general vehicle data, documents, linked owners, beneficiaries, check-passes, route permits, insurance, type cards, bluebooks, tare weight and audit history. This organization keeps the primary vehicle record readable while providing access to supporting records when needed.

3.3 Vehicle owners

Owner records can be searched and filtered by verification state, then opened, created or edited. The owner form changes according to whether the owner is an individual or a company: individuals supply citizenship information, while companies supply registration information. The detail screen groups profile information, bank details and linked vehicles.

Files selected in relevant owner, bank or vehicle forms are sent as multipart form data. The backend uploads them to Cloudinary, then associates returned file URLs and names with the corresponding record. This avoids storing image and document bytes inside the JSONB application state. Current upload limits are enforced by the API and are typically five megabytes per file; the exact fields and file count vary by endpoint.

3.4 Checkpoint staff and narration

Checkpoint user management supports staff records, profile changes and an independent active/inactive status. Province and district are linked choices: a district depends on the selected province. Narration settings hold text that is included on queue receipts, so staff can maintain reusable checkpoint wording separately from an individual entry.

3.5 Other vehicles, reports and QR

Other Vehicles holds Indian or unregistered vehicle records that are not in the registered-vehicle list. Reports include checkpoint-entry and detail-entry views with date and record filters, plus print or export actions. QR generation presents a checkpoint payload and can render it as a downloadable or printable code.

The checkpoint-entry report currently filters by checkpoint, user or local sangh text and by date range, and the API re-

turns paginated rows. Its totals are demo calculations--one vehicle per queue and the queue fee or a default fee--so the calculation should be checked against the authoritative accounting rules before being used for operational reconciliation.

4. Business rules represented in the UI

The frontend makes important dependencies visible close to the field where they matter:

- District selection is disabled until a province is chosen, because the district options depend on that province.
- Vehicle plate fields and the plate preview depend on plate type.
- Owner identification fields depend on individual versus company ownership.
- Queue routes must use distinct start and destination points.
- Check-in and end-journey are explicit operations that update the queue lifecycle.
- Pending queue approval and removal are separate from normal active queue actions.
- Date values use an ISO/AD representation at the API boundary while the queue form and receipt can display Bikram Sambat dates.
- Required fields and field-specific validation are handled in the client forms; the API also has a global request validation pipe, although not every controller applies complete business validation.

These are the rules confirmed by the current interface and code. Further rules--such as the authority for changing a completed transaction, fee calculation policy, or accounting reconciliation--need confirmation from the system owner before a production integration.

5. Architecture and implementation decisions

Feature-oriented frontend

The frontend uses the Next.js App Router for route composition. Each business area keeps its API module, hooks, types, schemas and components together under src/features. Shared UI and infrastructure live under src/shared, including the HTTP client, form fields, list state, tables, dialogs, pagination, toasts and application shell.

This structure keeps a queue-specific decision close to queue code rather than spreading it across a generic components folder. Shared components are reserved for repeated interaction patterns, such as an error state with retry or a table with loading, empty and pagination states.

Replaceable API boundary

Feature API modules call one Axios based apiFetch client. Pages and presentation components do not assemble server URLs themselves. The API base URL is selected by NEXT_PUBLIC_API_BASE_URL, and request/response types are kept with features and described in docs/API_CONTRACT.md.

This is useful for a later integration: a compatible service can be selected through environment configuration, and endpoint or serialization differences can be adapted in feature API modules. It does not mean that any arbitrary backend will work without an adapter; routes, field names, authentication and response envelopes must still match.

Server state and form state

TanStack Query owns fetched data, request lifecycle and cache invalidation after mutations. General queries stay fresh for 30 seconds, lookup data for five minutes, and the app avoids automatic refetch on every window focus. A request may be retried once for network or server failures; ordinary client errors are not retried. List hooks preserve the previous page while the next result loads to reduce visual flicker.

React Hook Form and Zod own local form state and validation. This keeps field errors close to their controls and avoids using a broad global state store for data that already has a clear owner: the query cache for server data and a form for draft input.

API behavior and feedback

The API is grouped into NestJS controllers for authentication, dashboard, lookup data, checkpoint users, narration, queues, vehicles, owners, other vehicles, reports and QR. A global interceptor wraps successful single values as { data }, leaves paginated responses with { data, meta }, and persists successful mutations before returning them. The exception filter returns a consistent { message, errors? } body.

The frontend maps these responses to loading, empty, validation, retry and toast states. Multipart requests are created only when a body contains a browser File; ordinary requests remain JSON. Browser credentials are included for cookie-based sessions.

6. Persistence, uploads and deployment boundary

Neon persistence

Without DATABASE_URL, the backend uses seeded in-memory collections. They reset when the backend process restarts. When DATABASE_URL is configured, the backend creates a tqms_app_state table and stores each application collection as a JSONB value. Existing stored collections take precedence over seed data; a collection is seeded only when it has no row. Successful writes persist the collection affected by that request before the API returns.

This approach is convenient for a demo and keeps the controller contract independent of the storage mechanism. It is not a normalized relational design. It does not provide relational constraints or fine-grained row-level transactions across domain entities. Session records are excluded from this Neon state store, so sessions are not durable across a restart. Production mode requires DATABASE_URL, but that alone does not make the persistence layer production grade.

Cloudinary media

The backend receives files and uploads them to Cloudinary using server-side credentials. Neon stores media metadata and URLs rather than binary file contents. Some multi-file owner uploads are issued concurrently to shorten request time, but the API still waits for Cloudinary before the record operation completes. Upload latency therefore depends on file size, network conditions and Cloudinary response time.

Current delivery URLs are standard Cloudinary URLs. Anyone who obtains a public URL may be able to retrieve the file. Identity documents, bank evidence or other sensitive files should use authenticated/private delivery and access-controlled URL generation before real personal data is stored.

Deployment boundary

The frontend can be hosted separately from the API. The deployed frontend must use the backend URL through NEXT_PUBLIC_API_BASE_URL, while the backend must allow the frontend origin through CORS and issue cookies with settings compatible with the deployment domains and HTTPS. Database and Cloudinary secrets must be configured only in the backend's private environment. Local demo credentials are for the seeded mock account and must not be reused as production credentials.

7. Performance and usability decisions

The implementation makes several modest, targeted performance choices:

- Query caching avoids repeating list and lookup requests while data remains fresh.
- Focus refetch is disabled to avoid extra network requests when staff move between tabs and return.
- Lookup data stays fresh longer than changing operational records.
- Paginated lists request bounded result sets; previous rows stay visible while a new page loads.
- Neon writes update only the affected collection instead of serializing every application collection on each change.
- A small PostgreSQL connection pool bounds backend database connections.
- Owner file uploads can run concurrently, and uploaded file bytes are kept out of the application database.

These choices reduce avoidable work, but they cannot eliminate cold starts on a free hosting tier, cross-region request time, database wake-up time, or the time needed to transfer and upload a large image. For production, performance should be measured using request timings and representative file sizes before adding more caching or background processing. Any background upload design must still clearly communicate upload and record status to the operator.

8. Known gaps and risks

The current implementation is a useful assessment/demo application, not a complete production replacement for the legacy TQMS service. Key gaps are:

1. Report business meaning: checkpoint-entry filters and pagination are implemented, but the demo total amount uses a simplified queue-fee/default calculation. Confirm reporting and accounting rules against the system owner before operational use.

2. Incomplete CRUD: some vehicle, owner and bank delete routes are absent; Other Vehicles supports list/create only; queues have lifecycle actions but no general edit/delete route.

3. Validation regression coverage: core checkpoint-user, vehicle, owner, queue and Other Vehicle write flows now return server-side field errors for invalid input. Keep the API smoke suite and frontend rules aligned as forms evolve.

4. Storage model: Neon stores collection snapshots as JSONB rather than normalized tables and domain repositories.

5. Session durability: session data is not stored in Neon, and the backend contains a development fallback session secret that must be replaced for deployment.

6. Media privacy: standard Cloudinary URLs are public delivery references unless private delivery is configured.

7. Failure and latency simulation: the mock API does not have configurable artificial delays or deterministic failure scenarios, even though these are useful for testing UI states.

8. Automated QA: there is no automated frontend/backend test suite or browser end-to-end suite configured. Manual smoke checks do not replace repeatable regression coverage.

9. Production controls: role-based authorization, audit requirements, rate limits, backups, monitoring and data-retention policy need a production review.

9. Recommended path to a real backend

The frontend should keep its feature APIs and typed contract as the integration seam. Before connecting to a real TQMS service, agree on the canonical models and status transitions with the backend owner, then map legacy endpoints and field names in the feature API layer. Preserve the queue lifecycle as explicit commands and verify receipt number generation, fee calculation, date semantics and report filters against authoritative rules.

For persistence, replace the JSONB collection store with migrations and normalized tables for users, vehicles, owners, queues, entries, payments and documents. Use database transactions for operations that change multiple related records, enforce uniqueness and foreign-key rules at the database boundary, and define a durable session or token strategy appropriate to the deployment.

For files, keep upload authorization on the server, validate file type and size from content as well as filename, store private assets where the data is sensitive, and issue short-lived access links only to authorized users. Add API contract tests, validation tests, role/permission tests and browser tests for the full queue workflow before any production rollout.

Finally, run a data migration and reconciliation plan separately from UI deployment. Demo seed data must not be merged into a real operational database. Roll out with monitoring and a rollback path, and confirm that users can reconcile every receipt and state transition with the existing system.

10. Source map

The main implementation references are:

- src/app/ -- App Router pages and application providers.
- src/features/queue/ -- queue forms, receipt preview, API and lifecycle hooks.
- src/features/vehicles/ and src/features/vehicle-owners/ -- vehicle and owner workflows.
- src/features/reports/ -- report filters, tables, print and export actions.

- src/shared/api/ -- shared request client, error mapping and query configuration.
- tqms-backend/src/ -- NestJS API controllers and shared middleware.
- tqms-backend/src/store/neon.service.ts -- Neon initialization and JSONB collection persistence.
- tqms-backend/src/store/cloudinary.service.ts -- Cloudinary upload integration.
- docs/API_CONTRACT.md -- current routes, request/response envelopes and query parameters.

Prepared for the frontend technical assessment. This report distinguishes observed workflow from the behavior implemented by the demo code; it is not a specification of the complete legacy backend.